

打刻申請一覧を管理者用コントロールパネルに統合

現時点でURLパスによる単一画面表示（`/admin/approval`）だったものを、既存の管理者用コントロールパネル内の一つの「タブ（部品）」として組み込むことで、管理者側の操作UXが向上します。

また、無機質な `accountId` ではなくユーザー名がカードに表示されることで、誰の申請なのかが一目で直感的に分かるようになります。

この仕様変更をスマートに実現するためのトラフィック（データの流れ）の考察と、具体的な修正コードを提示します。

ユーザー名を表示するためのトラフィック（仕組み）の考察

現在、打刻申請データ（`EntryExitCalendar`）には `registeredAccountId`（ユーザーID）しか記録されていません。管理者が直感的に誰の申請か理解できるようにするには、バックエンド側で「申請データを受け渡すタイミングで、アカウント情報テーブルからユーザー名を結合する」のが最もスマートです。

これを実現するために、バックエンド側で値を手渡すための専用の入れ物（DTO: Data Transfer Object）である `AttendanceRequestDTO.java` を用意し、Service層で名前を詰め替えてControllerからフロントへ返却するトラフィックを構築します。

【バックエンド内部のトラフィック】

1. Repository から「`isTimechangeDemand = 1`」のレコード（IDのみ保持）を掘り起こす
2. 各レコードの `accountId` を使って、UserAccount から「ユーザー名（Name）」を取得
3. 両方の情報を `AttendanceRequestDTO` にマージ（結合）してフロントエンドへ返却する

バックエンド（Java）側の修正コード

① DTOクラスの新設: AttendanceRequestDTO.java

dto パッケージに、フロントエンドへ送り返すための結合データを定義します。

```
package com.appspace.backend.dto;

import lombok.Data;
import java.time.LocalDate;

@Data
public class AttendanceRequestDTO {
    private String recordId;
    private String accountId;
    private String userName; // 新設：結合したユーザー名を入れる特等席
    private LocalDate recordDate;
    private String entryTime;
    private String exitTime;
```

```
private String tmpEntryTime;
private String tmpExitTime;
private String reason;
private String adminComment;
}
```

② Service層の修正: AdminAttendanceService.java

カレンダーレコードのリストをこの DTO のリストへ変換し、アカウント名（ユーザー名）を詰め込む処理へアップデートします。

```
package com.appspace.backend.service;

import com.appspace.backend.dto.AttendanceRequestDTO; // ★追加
import com.appspace.backend.entity.EntryExitCalendar;
import com.appspace.backend.repository.EntryExitCalendarRepository;
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
@Transactional
public class AdminAttendanceService {

    @Autowired
    private EntryExitCalendarRepository calendarRepository;

    // @Autowired
    // private UserAccountRepository userAccountRepository; // ★お使いのアカウントリポジトリを接続

    /**
     * 管理者ロジック1: 現在申請が届いているレコードに「ユーザー名」を結合して取得する
     */
    public List<AttendanceRequestDTO> getPendingTimechangeRequestsWithUserName() {
        List<EntryExitCalendar> rawList = calendarRepository.findByIsTimechangeDemandOrderByRecordDateAsc(1);

        return rawList.stream().map(record -> {
            AttendanceRequestDTO dto = new AttendanceRequestDTO();
            dto.setRecordId(record.getRecordId());
            dto.setAccountId(record.getRegisteredAccountId());
            dto.setRecordDate(record.getRecordDate());
            dto.setEntryTime(record.getEntryTime());
            dto.setExitTime(record.getExitTime());
            dto.setTmpEntryTime(record.getTmpEntryTime());
```

```

        dto.setTmpExitTime(record.getTmpExitTime());
        dto.setReason(record.getReason());
        dto.setAdminComment(record.getAdminComment());

        // データベースからアカウント情報を引き取り、ユーザー名をドッキングする
        String name = "不明なユーザー";
        String aId = record.getRegisteredAccountId();

        try {
            // リポジトリにメソッドを登録していない場合はこのタイミングで追加する
            Optional<UserAccount> account = userAccountRepository.findByAccountId(aId);

            if (account.isPresent()) {
                name = account.get().getUserName();
            } else {
                name = "社員 (" + record.getRegisteredAccountId() + ")";
            }
        } catch (Exception e) {
            // アカウントが見つからない場合のセーフティ
        }
        dto.setUserName(name);

        return dto;
    }).collect(Collectors.toList());
}

/* --- approveTimechange や rejectTimechange メソッドは以前のままでOK --- */
}

```

③ Controller層の修正: AdminAttendanceController.java

戻り値の型を、生のカレンダーエンティティから、先ほど作った DTO のリスト
(List<AttendanceRequestDTO>) へと変更します。

```
/**
 * 窓口1: 未承認の編集申請一覧（名前結合版）を一括取得する
 */
@GetMapping("/requests")
public ResponseEntity<List<com.appspace.backend.dto.AttendanceRequestDTO>> getPendingRequests() {
    // ユーザー名が紐付いた新しいサービスロジックを呼び出す
    return ResponseEntity.ok(adminService.getPendingTimechangeRequestsWithUserName());
}
```

3. フロントエンド（Vue.js）側の修正コード

これまで単体のビュー（画面）として機能していた `AdminAttendanceApprovalView.vue` を、コントロールパネルの1つの「子コンポーネント（タブ中身）」へとリファクタリングします。

① コンポーネントファイルの引っ越しとユーザー名表示への変更

ファイル内のテンプレートを一部書き換え、ヘッダーの表示を `req.accountId` から `req.userName` へと変更します。また、親のタブから呼び出される部品となるため、ファイル名を `AdminAttendanceApprovalView.vue` から `AdminAttendanceApprovalPanel.vue` のように変更して `src/components/` 配下等に移すと設計が綺麗になりますが、名前はそのままでも動作します。

変更箇所のコード（ `AdminAttendanceApprovalView.vue` 内の `template` ヘッダー部分）:

```
<div class="card-header">
  <span class="user-id">申請者: {{ req.userName }}</span>
  <span class="target-date">対象日: {{ formatDate(req.recordDate) }}</span>
</div>
```


② 「統括管理者用 コントロールパネル」への埋め込み

管理者のメイン画面（ `AdminUserListView.vue` ）を開き、新設する「打刻申請確認」タブを既存の2つのタブの間に滑り込ませます。

以下に、コントロールパネル側の実装イメージをご提示します。

```
<template>
  <div class="admin-control-panel">
    <h2>統括管理者用 コントロールパネル</h2>

    <div class="tabs-header">
      <button
        @click="currentTab = 'accounts'"
        :class="{ active: currentTab === 'accounts' }"
      >
        アカウント一覧・退会管理
      </button>
```

```
<button
  @click="currentTab = 'requests'"
  :class="{ active: currentTab === 'requests' }"
>
  打刻申請確認
</button>

<button
  @click="currentTab = 'modes'"
  :class="{ active: currentTab === 'modes' }"
>
  打刻モード一括制御
</button>
</div>
<div class="tab-content">

  <!-- 追加分 -->
  <div v-if="currentTab === 'requests'">
    <AdminAttendanceApprovalView />
  </div>

</div>
</div>
</template>

<script setup>
import { ref } from 'vue';
import AdminAttendanceApprovalView from '../components/AdminAttendanceApprovalView.vue';
```

```
// 初期表示するタブの定義 ('accounts' からスタート)
const currentTab = ref('accounts');
</script>

<style scoped>
/* タブ全体のスタイリング（既存のものに調和させてください） */
.tabs-header {
  display: flex;
  border-bottom: 2px solid #cfd8dc;
  margin-bottom: 25px;
  gap: 5px;
}
.tabs-header button {
  padding: 12px 24px;
  background: #f5f7f8;
  border: 1px solid #cfd8dc;
  border-bottom: none;
  border-radius: 6px 6px 0 0;
  cursor: pointer;
  font-weight: bold;
  color: #546e7a;
  transition: all 0.2s;
}
.tabs-header button:hover {
  background: #eef;
}
/* アクティブ（選択中）なタブのスタイル */
.tabs-header button.active {
  background: #37474f;
  color: white;
  border-color: #37474f;
}
.tab-content-body {
  background: #ffffff;
  padding: 10px;
  border-radius: 0 0 6px 6px;
}
</style>
```

連動と表示の確認ポイント

ソースコードをそれぞれ適用後、サーバーを立ち上げて管理者画面（コントロールパネル）へアクセスしてみてください。

1. タブの並び順の確認:

「アカウント一覧・退会管理」と「打刻モード一括制御」の真ん中に「打刻申請確認」というタブが美しく整列しているか確認します。

2. コンテンツの動的切り替え:

真ん中のタブを押した瞬間に、URLが切り替わることなく、下のエリアにフワッといつもの打刻申請カードの一覧が展開されるか確認します。

3. ユーザー名の確認:

展開された各カードの左上のヘッダーが、無機質な `USER001` といったID表記から、JPA結合によって引き出された `申請者: 社員 (USER001)` や実際の氏名テキストに置き換わっていれば、フロント・バックエンド双方の改修が完全大成功となります！

これで管理者機能もバラバラの画面ではなく、一つのコントロールパネルに集約されました。